CareFlow

Healthcare Appointment & Follow-up Manager

SYSTEM DESIGN WRITE-UP

Booking integrity	PostgreSQL unique slot row + atomic insert
Failure isolation	Transactional outbox + channel-specific retries
AI continuity	Gemini, secondary model, cache, rules, local safety template

Architecture

CareFlow separates authentication, doctor scheduling, appointments, AI, and notifications behind a JWT-validating API gateway. Each service owns a PostgreSQL database and communicates through narrow REST contracts. This reduces schema coupling and lets reliability-sensitive appointment logic remain independent of external AI, email, and calendar providers.

1. Double-booking prevention

Availability shown in the UI is advisory; correctness is enforced when a hold is created. slot_reservations has a PostgreSQL unique constraint on (doctor_id, start_at). The appointment service validates the doctor's working hours and leave state, then executes INSERT ... ON CONFLICT DO NOTHING. When two requests race, PostgreSQL serializes the index conflict: one inserts the reservation and receives a token, while every loser receives HTTP 409. This works across threads and multiple service replicas; it does not depend on an in-memory lock.

Confirmation locks the reservation row with a pessimistic database lock, verifies the patient, expiry, and unused token, rechecks doctor availability, writes the appointment, and marks the reservation confirmed in one transaction. Replayed tokens cannot create another appointment. Cancellation deletes the reservation so the slot can be booked again. Rescheduling consumes a new hold and releases the old reservation within one transaction. Appointment rows also use an optimistic version for conflicting updates.

2. Slot hold mechanism

A hold lasts five minutes. It contains a random single-use token, patient ID, doctor ID, start time, and expiry. Expired unconfirmed rows are removed before new holds and by a scheduled cleanup. Confirmed reservations receive the appointment ID and a non-expiring marker. The patient never confirms using only a time value; the token proves ownership of the exact reserved row. If the hold expires, the API returns 409 and asks the patient to choose again.

Parallel holds	Unique index selects one winner	201 / 409
Token replay	Locked row already confirmed	409
Expired hold	Expiry check + cleanup	409; select again
Cancellation	Reservation deleted	Slot reusable

3. Doctor leave conflicts

The admin creates leave in the doctor service, which synchronously asks the appointment service to resolve that doctor/date before its own transaction commits. If conflict processing cannot run safely, leave creation rolls back and returns a visible service-unavailable error instead of leaving inconsistent state. The appointment service selects confirmed appointments in the clinic timezone, marks each cancelled with a specific reason, deletes its slot reservation, and writes one notification event per appointment. Patients can then immediately book another clinician or date. The event instructs Calendar delivery to delete existing events for both participants.

4. Notification failure handling

Booking must not fail because email or Google is down. Therefore appointment changes and an outbox event are committed in the same database transaction. A worker publishes pending events to the notification service with a deterministic idempotency key. Failures use exponential backoff and remain queryable; repeated delivery cannot create duplicate jobs.

The notification service creates independent patient-email, doctor-email, patient-calendar, and doctor-calendar jobs. One failed channel does not block another. Jobs persist status, attempt count, next attempt, and last error, and retry to a bounded maximum. SMTP can run in explicit dry-run mode for development. Calendar tokens refresh automatically; event IDs are stored per participant and appointment so reschedule uses update and cancellation uses delete. Medication plans store the clinician-entered interval and end date, and a scheduler emits reminders when due.

5. Graceful AI degradation

AI follows the same graceful-degradation principle. The chain is primary Gemini, retry, secondary Gemini model, cached validated output, deterministic rules, and finally a local appointment-service safety template if the AI service is unreachable. Structured responses are validated and stored, medication text is never invented, and every output is labelled for clinician review. Thus AI improves preparation but is never part of the booking integrity boundary.

Design result	Appointment consistency depends only on PostgreSQL transactions. External providers improve the experience but cannot corrupt booking state.